Proiectare Logică

I CB: Mariana Mocanu mariana.mocanu@upb.ro
I CC: Anca Morar anca.morar@upb.ro
I CA & CD: Costin Chiru costin.chiru@upb.ro

Curs 6: Circuite combinationale MSI (multiplexoare, demultiplexoare codificatoare, decodificatoare)

Circuite combinaționale

- Circuite integrate pe scară mică și medie
- Tipuri de circuite:
  - Porți
  - Multiplexoare / demultiplexoare
  - Codificatoare / decodificatoare
  - Comparatoare
  - Convertoare
  - Sumatoare
- Pe măsură ce coborâm, crește complexitatea lor



$$Q^t = f(I_1^t, ..., I_n^t)$$

- Ne interesează dispozitivele din punct de vedere logic, dar ne preocupă și partea de implementare → Câteva informații pe partea de inginerie

Porți (Gates)

- Inversor
- Repetor
- Poarta ȘI *(AND)*
- Poarta SAU *(OR)*
- Poarta ȘI-NU *(NAND)*
- Poarta SAU-NU *(NOR)*
- Poarta SAU-EXCLUSIV *(XOR)*
- Poarta SAU-EXCLUSIV-NEGAT *(XNOR)*

- 2, 3, 4, 8 intrări
- Intrările nefolosite nu se lasă în aer, se conectează între ele sau se leagă la elementul neutru
- Timp de propagare
- Timp de tranziție
- Fan-out

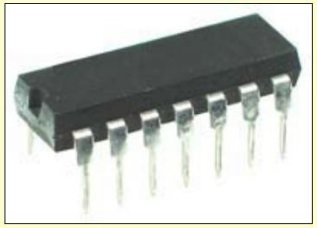
Poarta NAND cu 2 intrări

Cum arată + care e codul

Ce face + unele caracteristici constructive

7400 - 7400 Quad 2-Input NAND Gate Datasheet

Photograph

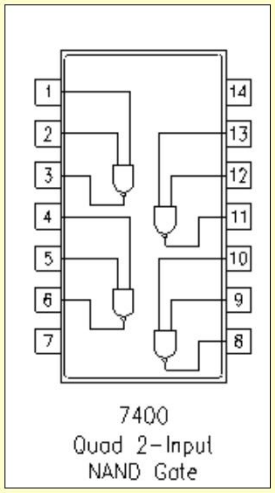


Features

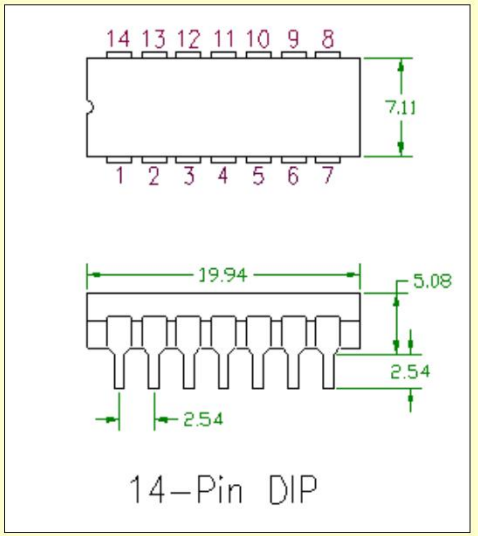
- Four Independent 2-Input NAND Gates
- Outputs Directly Interface to CMOS, NMOS and TTL
- Large Operating Voltage Range
- Wide Operating Conditions
- Not Recommended for New Designs Use 74LS00

Dimensiunile piesei + cheița

Pin Layout



Dimensional Drawing



Funcționarea + ce reprezintă pinii (7 – GND, 14 – VCC) + structura constructivă

SNx400, SNx4LS00, and SNx4S00 Quadruple 2-Input Positive-NAND Gates

1 Features

- Package Options Include:
  - Plastic Small-Outline (D, NS, PS)
  - Shrink Small-Outline (DB)
  - Ceramic Flat (W)
  - Ceramic Chip Carriers (FK)
  - Standard Plastic (N)
  - Ceramic (J)
- Also Available as Dual 2-Input Positive-NAND Gate in Small-Outline (PS) Package
- Inputs Are TTL Compliant; V_{IH} = 2 V and V_{IL} = 0.8 V
- Inputs Can Accept 3.3-V or 2.5-V Logic Inputs
- SN5400, SN54LS00, and SN54S00 are Characterized For Operation Over the Full Military Temperature Range of –55°C to 125°C

2 Applications

- AV Receivers
- Portable Audio Docks
- Blu-Ray Players
- Home Theater
- MP3 Players or Recorders
- Personal Digital Assistants (PDAs)

3 Description

The SNx4xx00 devices contain four independent, 2-input NAND gates. The devices perform the Boolean function $Y = \overline{A \cdot B}$ or $Y = \overline{A} + \overline{B}$ in positive logic.

Device Information(1)

PART NUMBER	PACKAGE	BODY SIZE (NOM)
SN74LS00DB	SSOP (14)	6.20 mm × 5.30 mm
SN7400D, SN74LS00D, SN74S00D	SOIC (14)	8.65 mm × 3.91 mm
SN74LS00NSR	PDIP (14)	19.30 × 6.35 mm
SNJ5400J, SNJ54LS00J, SNJ54S00J	CDIP (14)	19.56 mm × 6.67 mm
SNJ5400W, SNJ54LS00W, SNJ54S00W	CFP (14)	9.21 mm × 5.97 mm
SN54LS00FK, SN54S00FK	LCCC (20)	8.89 mm × 8.89 mm
SN7400NS, SN74LS00NS, SN74S00NS	SO (14)	10.30 mm × 5.30 mm
SN7400PS, SN74LS00PS	SO (8)	6.20 mm × 5.30 mm

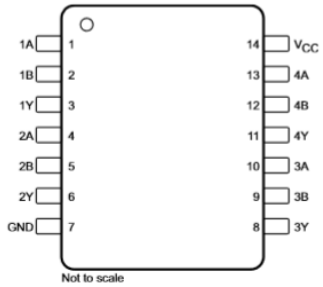
(1) For all available packages, see the orderable addendum at the end of the data sheet.

Logic Diagram, Each Gate (Positive Logic)



5 Pin Configuration and Functions

SN5400 J, SN54xx00 J and W, SN74x00 D, N, and NS
SN74LS00 D, DB, N, and NS Packages
14-Pin CDIP, CFP, SOIC, PDIP, SO, or SSOP
Top View



Not to scale

Pin Functions

PIN					I/O	DESCRIPTION
NAME	CDIP, CFP, SOIC, PDIP, SO, SSOP	SO (SN74xx00)	CFP (SN5400)	LCCC		
1A	1	1	1	2	I	Gate 1 input
1B	2	2	2	3	I	Gate 1 input
1Y	3	3	3	4	O	Gate 1 output
2A	4	6	6	6	I	Gate 2 input
2B	5	7	7	8	I	Gate 2 input
2Y	6	5	5	9	O	Gate 2 output
3A	10	—	9	13	I	Gate 3 input
3B	9	—	10	14	I	Gate 3 input

Copyright © 1983–2017, Texas Instruments Incorporated

Descrierea pinilor, în care se spune fiecare ce face

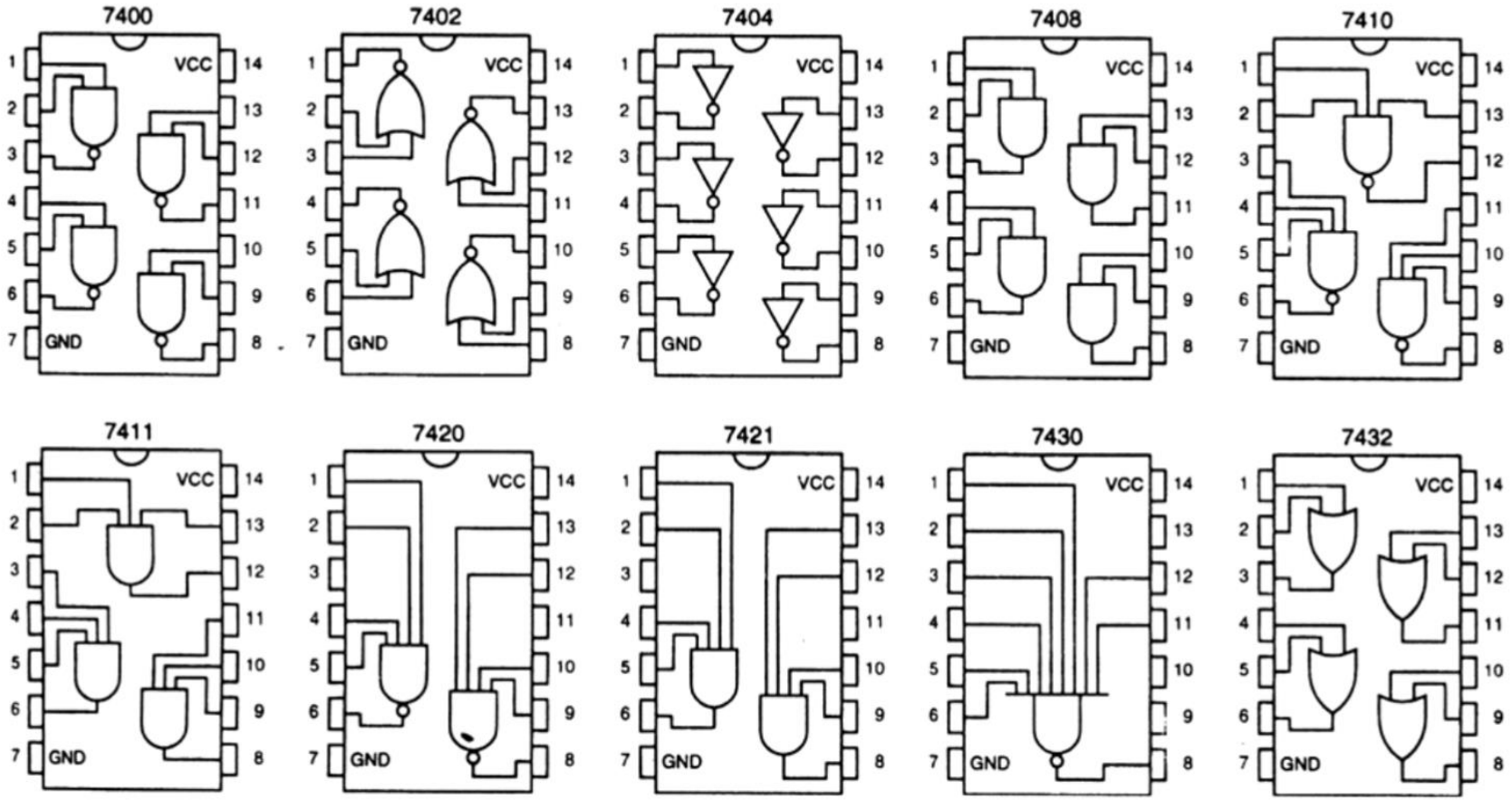
6.5 Electrical Characteristics: SNx400

over operating free-air temperature range (unless otherwise noted)

PARAMETER	TEST CONDITIONS		MIN	TYP	MAX	UNIT
V_{IK}	V_{CC} = MIN and I_I = –12 mA				–1.5	V
V_{OH}	V_{CC} = MIN, V_{IL} = 0.8 V, and I_{OH} = –0.4 mA		2.4	3.4		V
V_{OL}	V_{CC} = MIN, V_{IH} = 2 V, and I_{OL} = 16 mA			0.2	0.4	V
I_I	V_{CC} = MAX and V_I = 5.5 V				1	mA
I_{IH}	V_{CC} = MAX and V_I = 2.4 V				40	µA
I_{IL}	V_{CC} = MAX and V_I = 0.4 V				–1.6	mA
I_{OS}	V_{CC} = MAX	SN5400	–20		–55	mA
		SN7400	–18		–55	
I_{CCH}	V_{CC} = MAX and V_I = 0 V			4	8	mA
I_{CCL}	V_{CC} = MAX and V_I = 4.5 V			12	22	mA

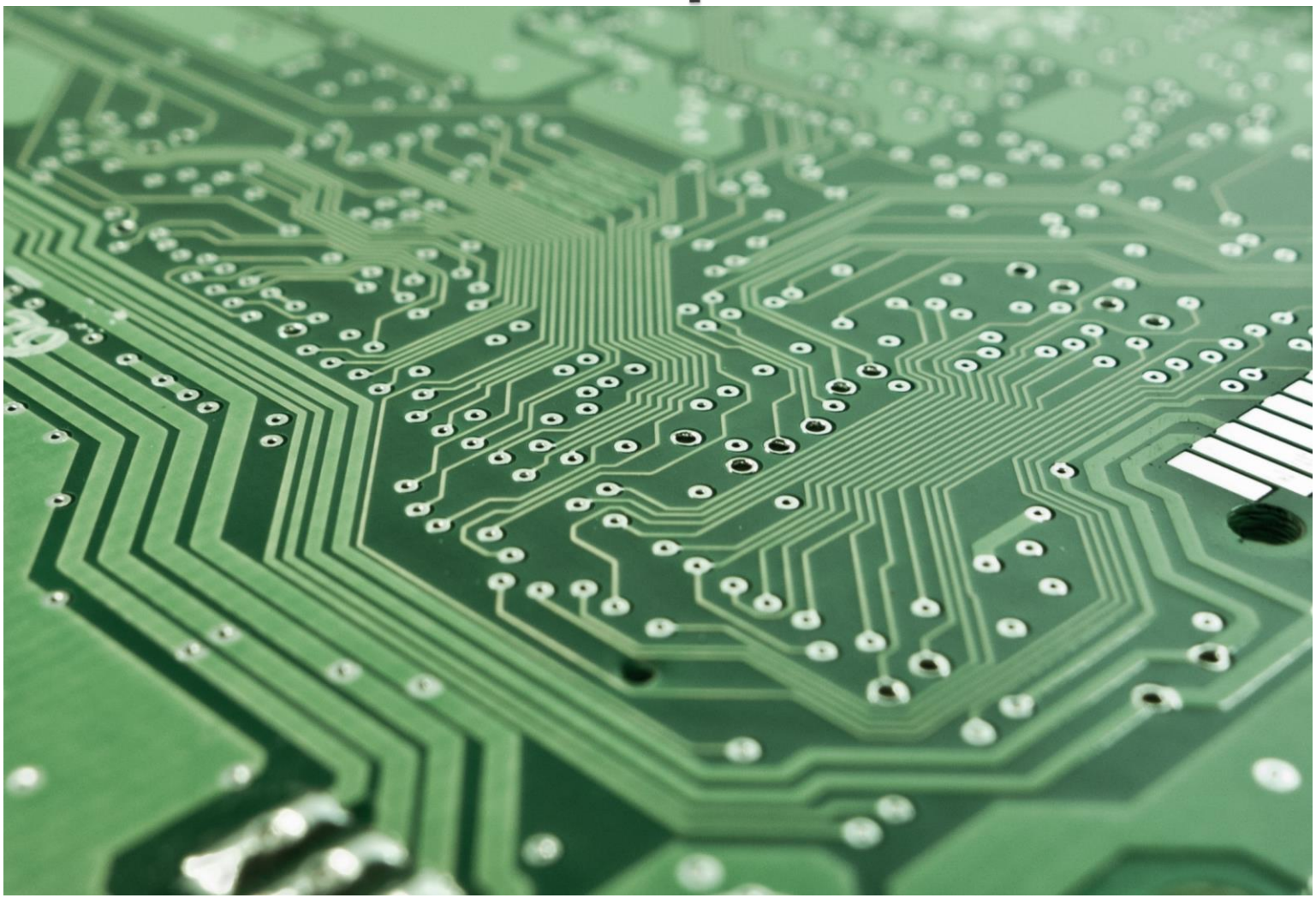
Precizări în funcție de varianta constructivă legate de curenți de in / out, tensiune de alimentare etc, ce depind de modul de operare

Capsule (chip/dip-uri) cu porți logice

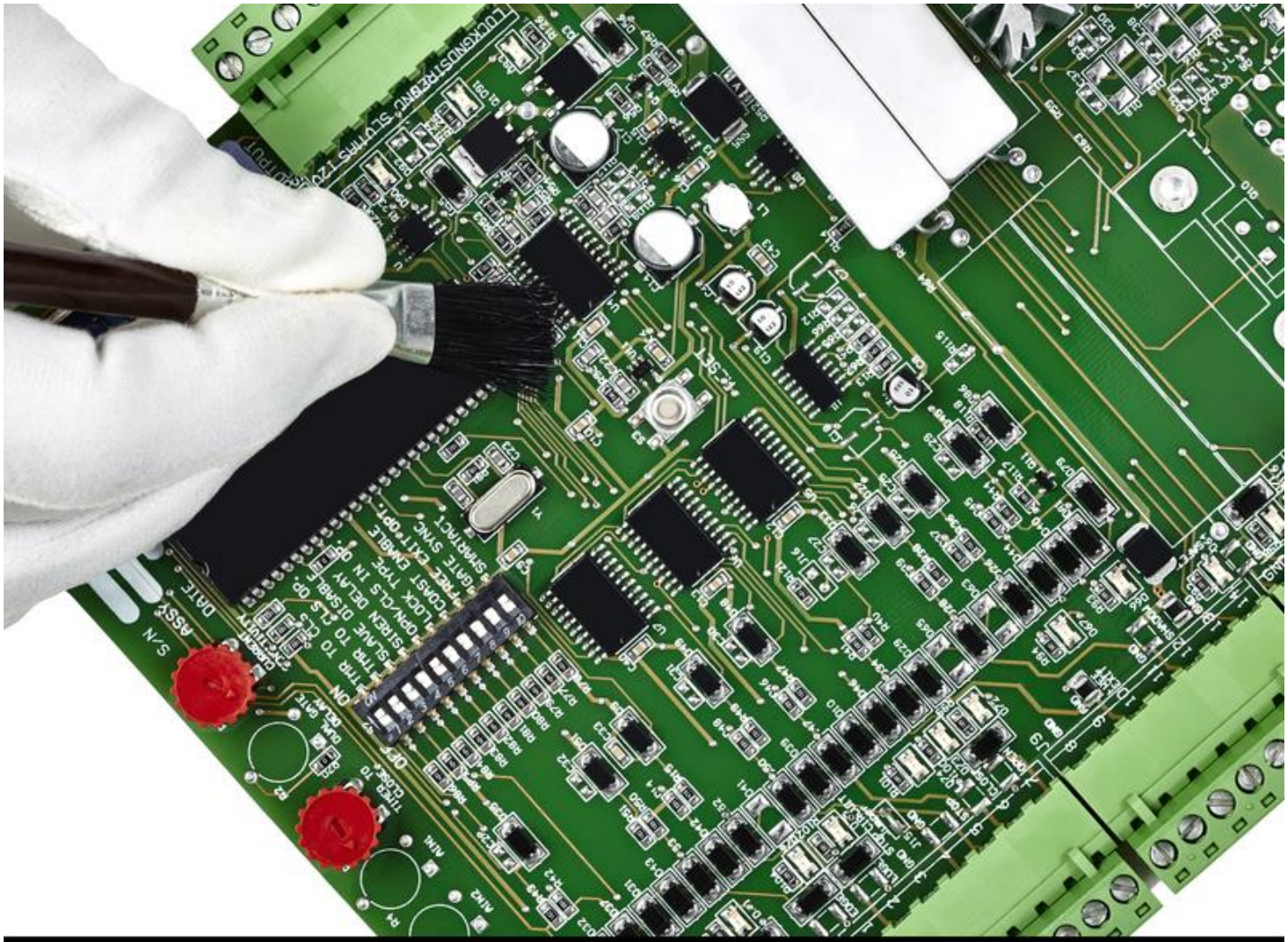


- Mai multe tipuri de circuite de care am făcut până acum – porți
- Fiecare are un număr – standardizat (terminațiile sunt cunoscute – 400, 402 ...) + ne spun informații despre varianta constructivă – are L/S/7/5
- Toate sunt pe capsule cu 14 pini, dar există și capsule cu 16/24/64 pini

Circuit cablat imprimat



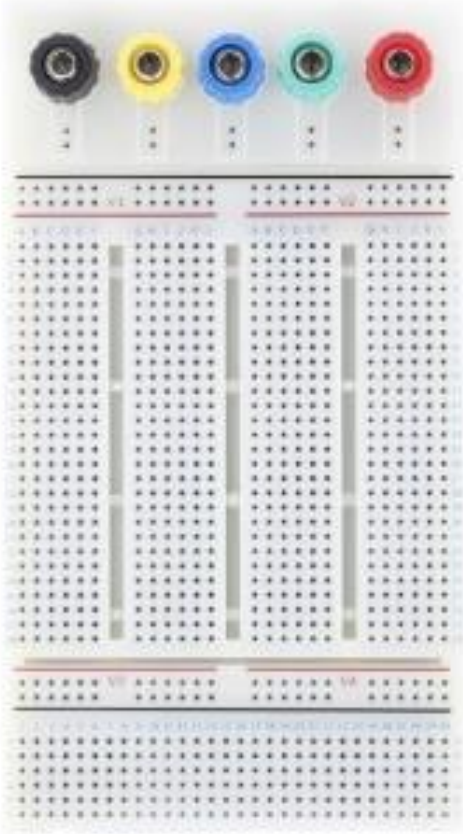
- Condiții constructive care guvernează buna funcționare a circuitului
  - nu e suficient să fie doar logic corect, ci trebuie să fie și constructiv

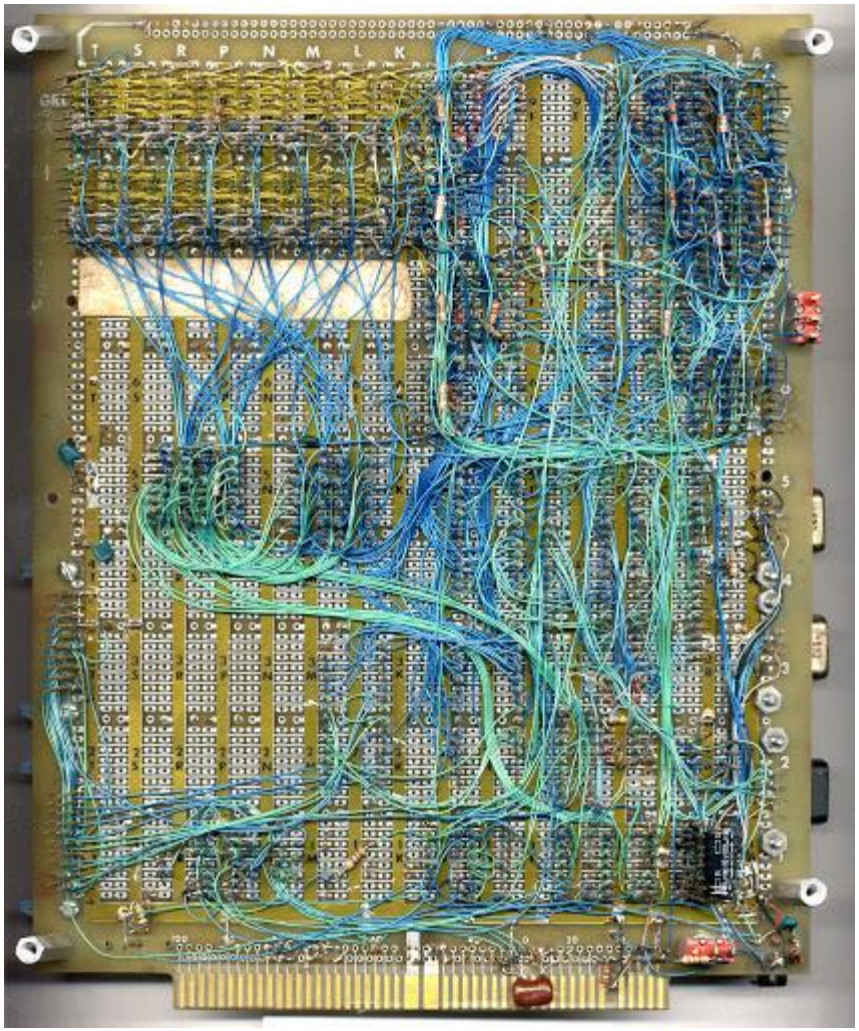


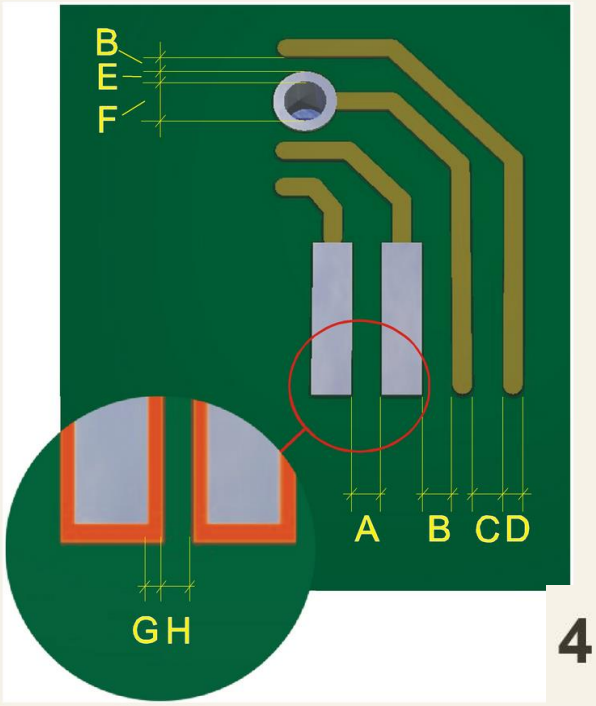
ADVANCEMENT & FUTURE OF PRINTED CIRCUIT BOARD

How to start?

- Plăci special construite pentru acest lucru (breadboard)
- Au găuri pentru a introduce / scoate circuite mai mari sau mai mici
- Pot fi alimentate la borne
- Fac legătura între pini







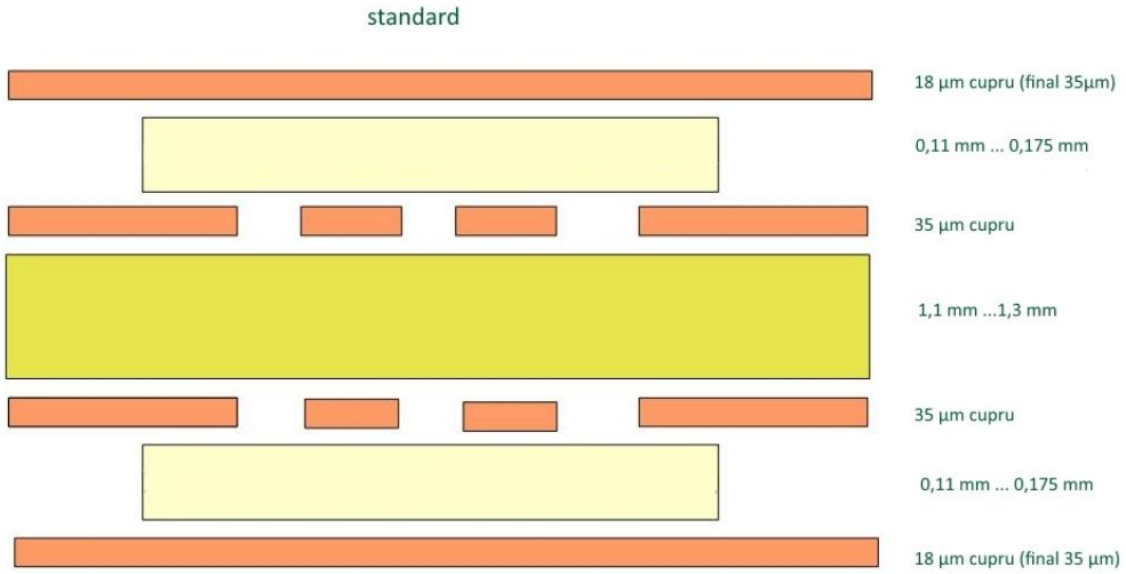
- A: pad-pad 6 mil (0,15mm)
- B: pad-traseu 6 mil (0,15mm)
- C: traseu-traseu 6 mil (0,15mm)
- D: latime traseu 6 mil (0,15mm)
- E: restring 6 mil (0,15mm)
- F: diametru gaura 12 mil (0,3mm)
- G: masca de lipire sa fie cu **2 mil** (0,05mm) mai mare decat padul
- H: masca mai mica de 5 mil (0,1 mm) nu se garanteaza!

- Se acoperă cu Cu și apoi se corodează ca să rămână doar traseele care îmi trebuie
- Pot avea 1, 2, 4, 8, 16 straturi

4 straturi

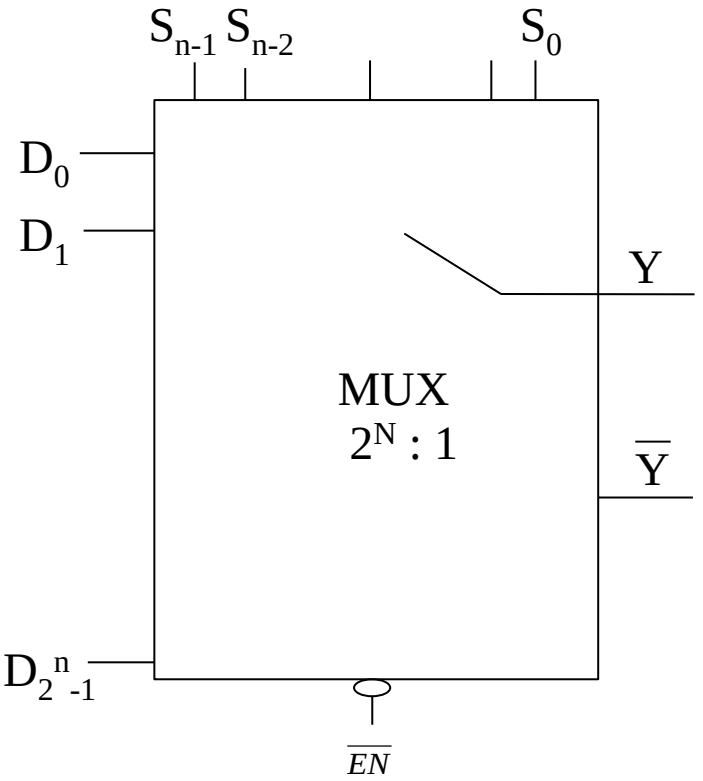
- Modul în care construiesc un cablaj este parte din proiectarea unei scheme

- La circuitele standard alimentarea e în interior → există locuri bine precizate pe unde se pot pune pini, pentru că dacă nu îi punem unde trebuie, facem scurt și se strică placa



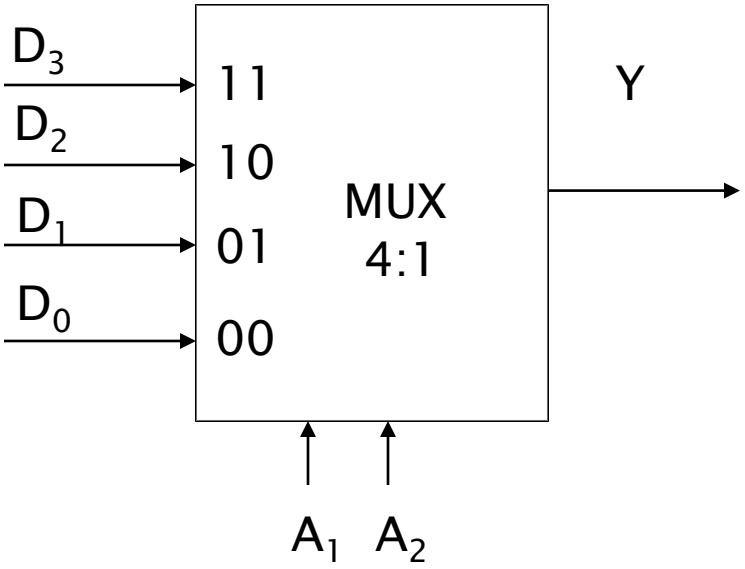
Multiplexoare

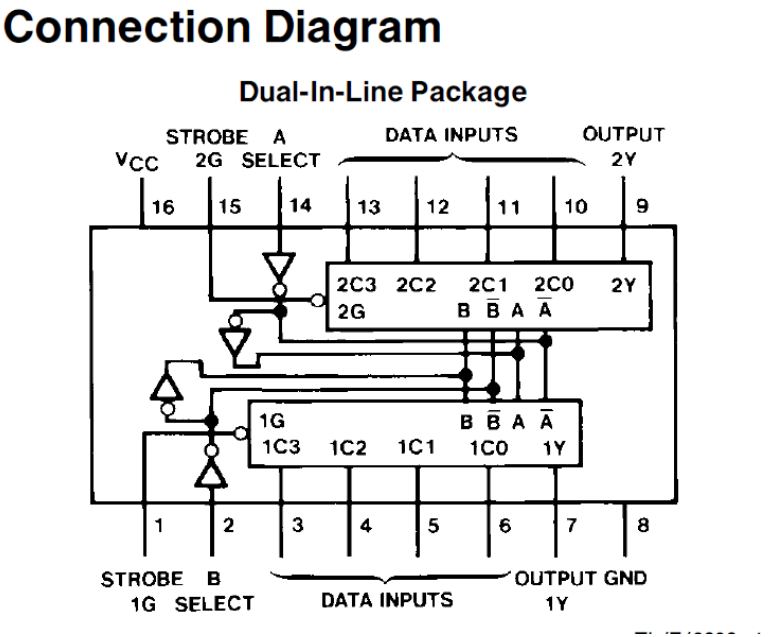
- Circuite integrate pe scara medie
- Circuite combinaţionale cu n intrări de selecție, 2^n intrări de date, o ieșire și eventual ieșirea negată, și o intrare de enable.
- Circuite care selectează pe baza unui cod, una dintre intrări, a cărei valoare o trimite la ieșire
- Fiecare intrare este identificată printr-un cod
- Ieșirea $Y = D_k$, $k = (S_{n-1}S_{n-2}...S_0)_{10}$
- Poate fi folosit pentru a genera funcții logice
- MUX 2/4/8/16:1



- **Observație**: Pentru a se implementa o funcție cu multiplexoare, acea funcție nu se mai minimizează in prealabil!

Multiplexorul 4:1

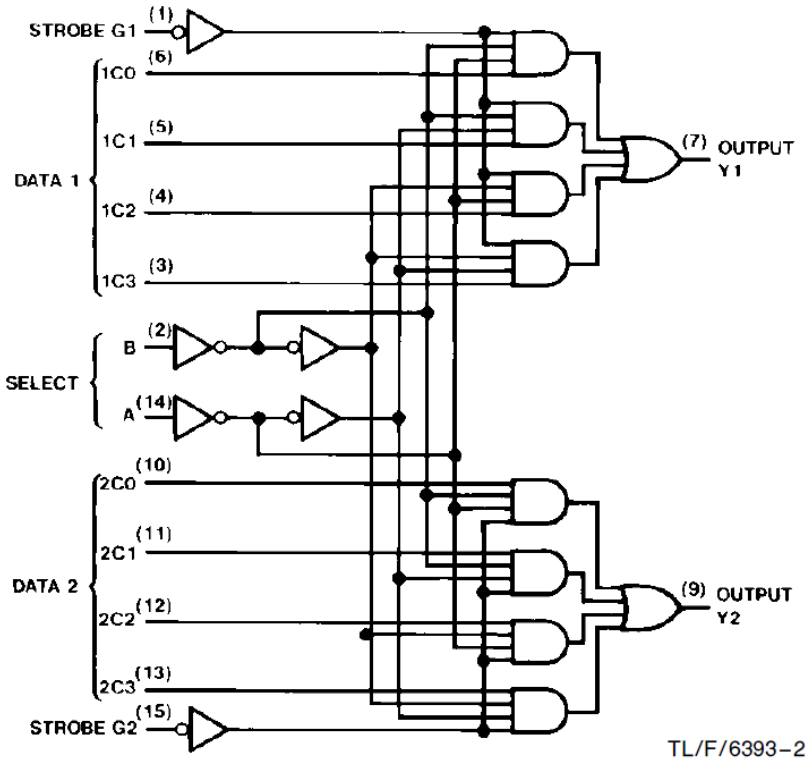




- Trebuie puse combinațiile de valori asignate intrărilor în interiorul MUX
- Cip cu 16 pini = 2 MUX 4:1 care depind de aceleași variabile de selecție – A & B. Strobe A & B sunt Enable-urile MUX-urilor

Multiplexorul 4:1 – funcționare

Logic Diagram



TL/F/6393–2

Function Table

Select Inputs		Data Inputs				Strobe	Output
B	A	C0	C1	C2	C3	G	Y
X	X	X	X	X	X	H	L
L	L	L	X	X	X	L	L
L	L	H	X	X	X	L	H
L	H	X	L	X	X	L	L
L	H	X	H	X	X	L	H
H	L	X	X	L	X	L	L
H	L	X	X	H	X	L	H
H	H	X	X	X	L	L	L
H	H	X	X	X	H	L	H

Select inputs A and B are common to both sections.

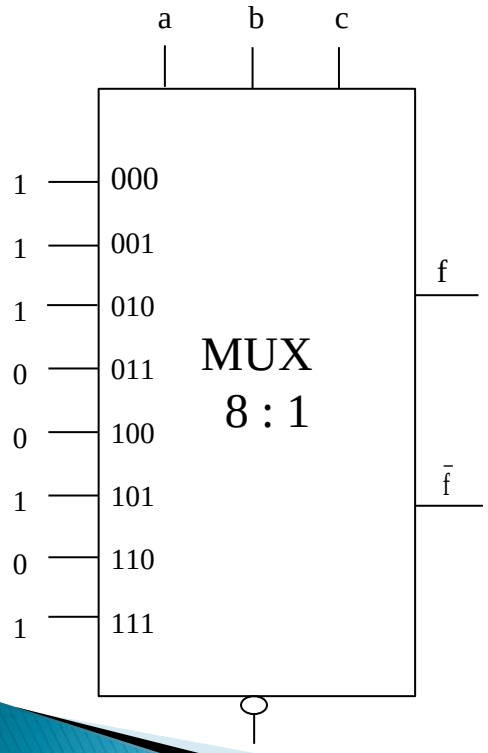
H = High Level, L = Low Level, X = Don't Care

L = 0, H = 1, X = indiferent

- Pornind de la tabelul de adevăr, prin minimizare, se ajunge la schema logică a circuitului → în desen avem 2 circuite (2 MUX pe capsulă), dar tabelul corespunde unei singure părți

Multiplexoare – exemplu (1)

- f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c
- Să se implementeze funcția folosind un multiplexor 8:1



- Cum rezolvăm?
- Scriem forma generală a unei funcții ca sumă de mintermi:
- $f(a,b,c) = c_0 * !a!b!c + c_1 * !a!bc + \ldots + c_7 * abc$
- De ce am scris funcția așa?
- Ca să putem identifica coeficienții și să îi punem pe intrările MUX-ului

Multiplexoare – conectare în cascadă (1)

- f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c
- Să se implementeze funcția folosind multiplexoare 2:1
- Cum se face?
- Se descompune un MUX mai mare în mai multe MUX-uri mai mici
- Numărul total de intrări în primul nivel al MUX-urilor legate în cascadă = numărul de intrări din MUX-ul mare = numărul de linii din tabelul de adevăr.
- Dimensiunea MUX-ului impune numărul de intrări în MUX, dar și numărul de variabile de selecție → MUX-uri mai mici = mai puține variabile de selecție

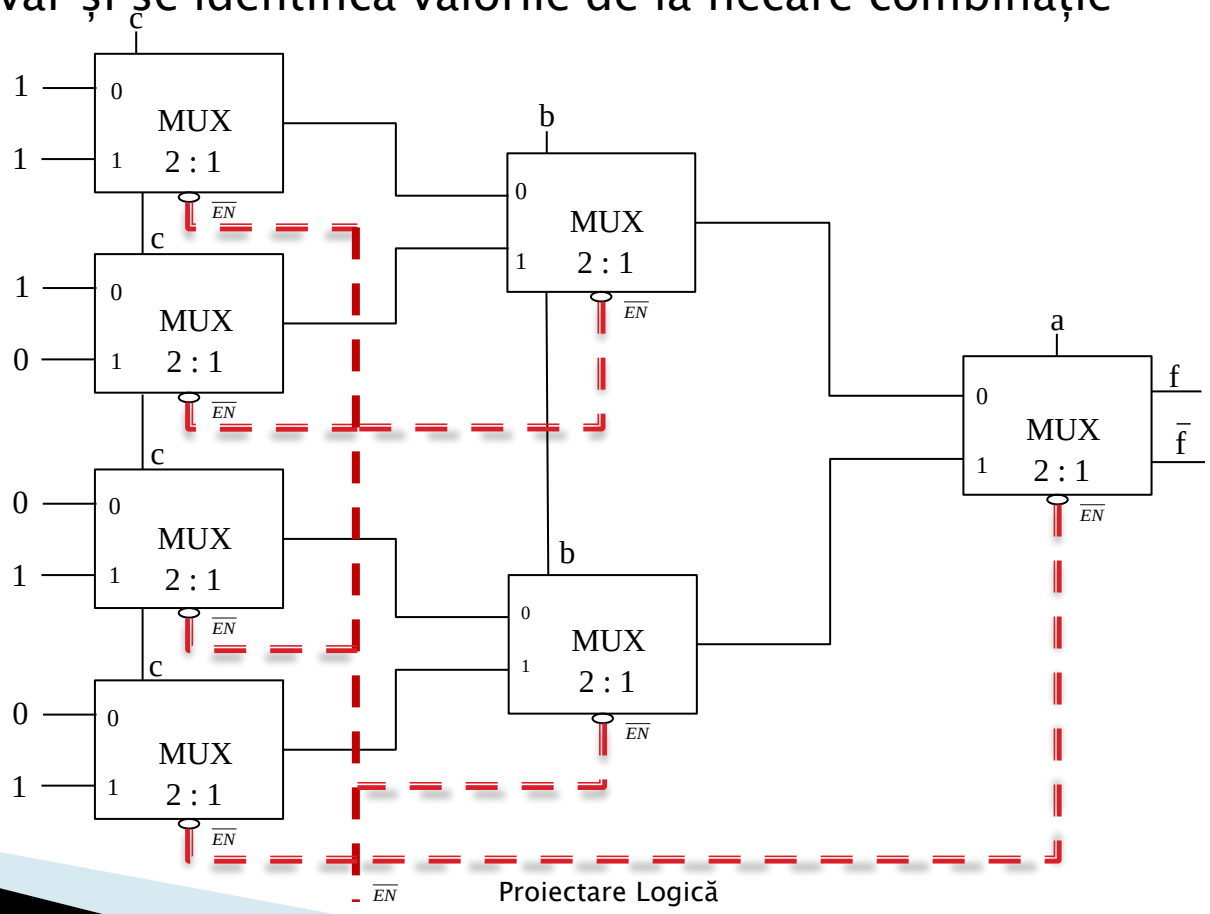
Multiplexoare – conectare în cascadă (2)

- Ex: o funcție ce depinde de n variabile de selecție poate fi implementată prin:
  - 1 MUX 2^n:1 sau
  - 2 MUX 2^{n-1}:1 (n-1 din variabilele de selecție) + 1 MUX 2:1 (ultima variabilă de selecție) care decide care dintre ieșirile celor două MUX-uri va fi folosită pentru a stabili valoarea finală a funcției sau
  - 4 MUX 2^{n-2}:1 , n >= 4 (n-2 din variabilele de selecție) + 1 MUX 4:1 (ultimele 2 variabile de selecție) sau
  - 4 MUX 2^{n-2}:1, n >= 4 (n-2 din variabilele de selecție) + 3 MUX 2:1 cuplate în cascadă (2 MUX 2:1 au ca intrări ieșirile din cele 4 MUX 2^{n-2}:1, urmând ca ultimul MUX 2:1 să aibă ca intrări ieșirile din cele 2 MUX 2:1) sau ...

Multiplexoare – conectare în cascadă (3)

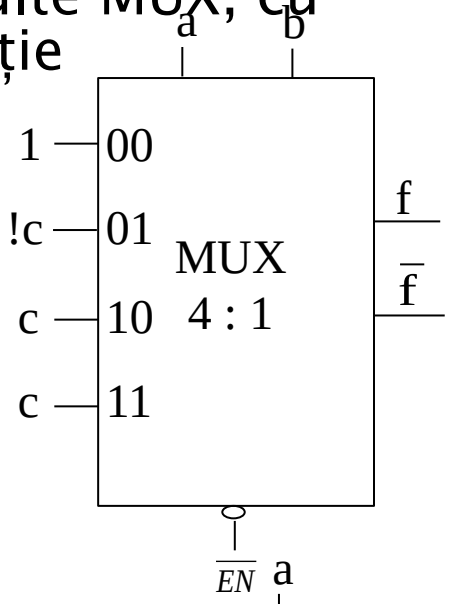
- f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c
- Câte MUX 2:1 sunt necesare?
- 4 + 2 + 1
- Cum se completează?
- Se face tabelul de adevăr și se identifică valorile de la fiecare combinație

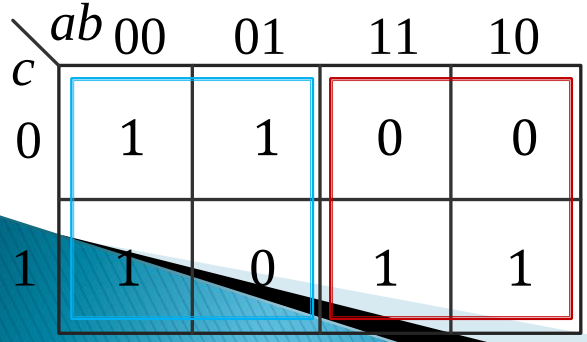
a	*b*	*c*	*f*
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

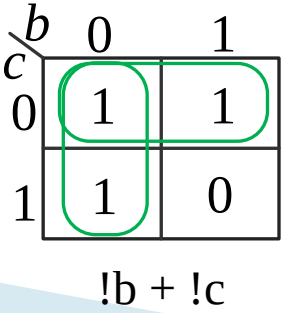


Multiplexoare – multiplexarea intrărilor (1)

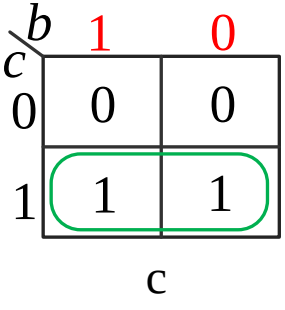
- E greu de realizat asta?
- Nu – iau unul sau mai multe MUX, identific coeficienții și aplic pe intrări valori elementare.
- Problemă: cu cât e mai mare un / sunt mai multe MUX, cu atât e mai scump →implementez aceeași funcție cu un singur MUX 4:1
- Se obține același rezultat dacă pun a, b sau a, c sau b, c pe condiții?
- f(a, b, c) = a!bc + !a!bc + abc + !a!b!c+!ab!c
- f(a, b) = c_0*!a!b + c_1*!ab+c_2*a!b+c_3*ab
- Identific coeficienții și completez MUX
- Pot trece și la MUX 2:1 în același fel, dar este mai complicat! → varianta mai simplă e să folosim diagramele Karnaugh



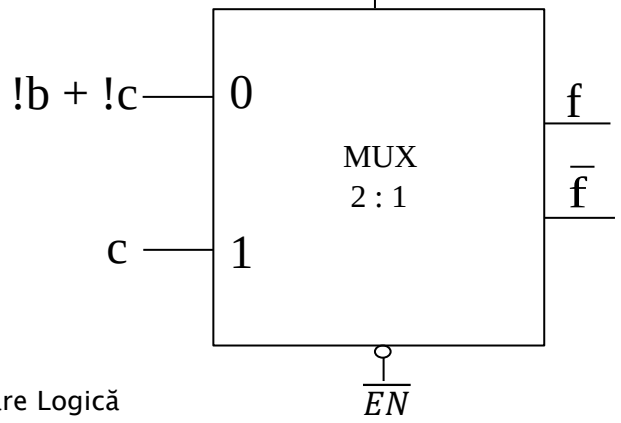




!b + !c



c



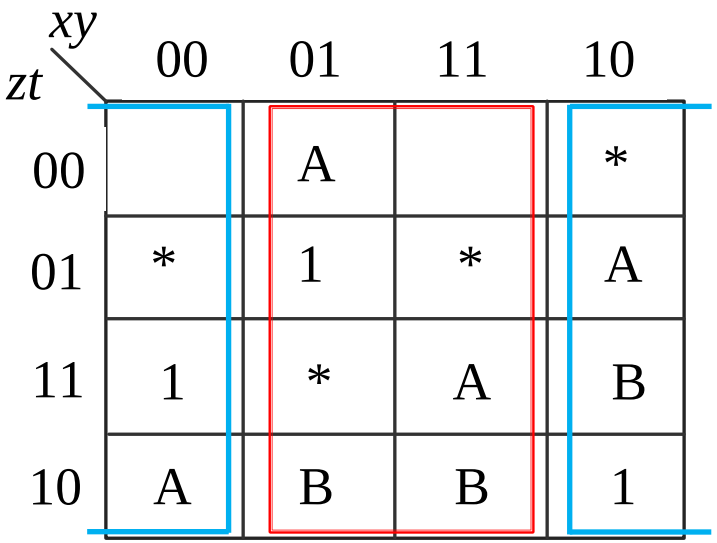
Terminologie

- Spațiul de adresare este dat de variabilele de selecție ale multiplexorului
  - în exemplul dat a fost {a, b, c}; {a, b}; {a}
- Spațiul de definiție al funcției este dat de variabilele de care depinde funcția
  - în exemplu dat {a, b, c}
- Dacă spațiul de adresare este mai mic decât spațiul de definire al funcției, diferența dintre cele două o constituie variabilele reziduu
  - în exemplu dat {c}; {b, c};

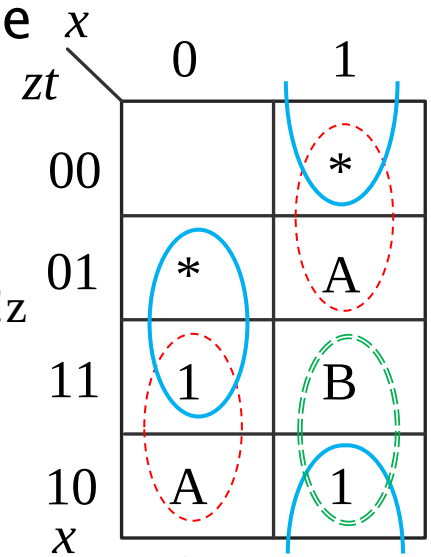
x	y	z	t	f
0	0	0	0	**0**
0	0	0	1	*
0	0	1	0	**A**
0	0	1	1	**1**
0	1	0	0	**A**
0	1	0	1	**1**
0	1	1	0	**B**
0	1	1	1	*
1	0	0	0	*
1	0	0	1	**A**
1	0	1	0	**1**
1	0	1	1	**B**
1	1	0	0	**0**
1	1	0	1	*
1	1	1	0	**B**
1	1	1	1	**A**

Multiplexoare – multiplexarea intrărilor (2)

- Implementați funcția folosind un singur MUX 2:1, având y ca variabilă de selecție

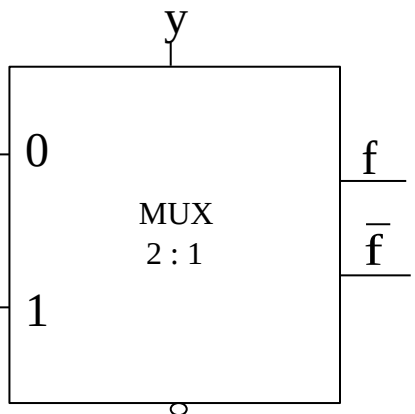


!x*t + x*!t +
A*!x*z + A*x*!z
+ B*x*z

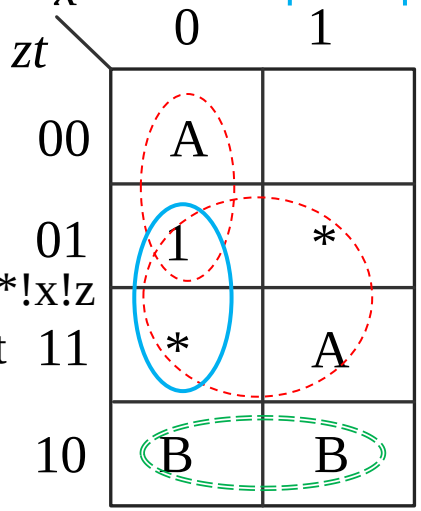


!x*t + x*!t +
A*!x*z + A*x*!z
+ B*x*z

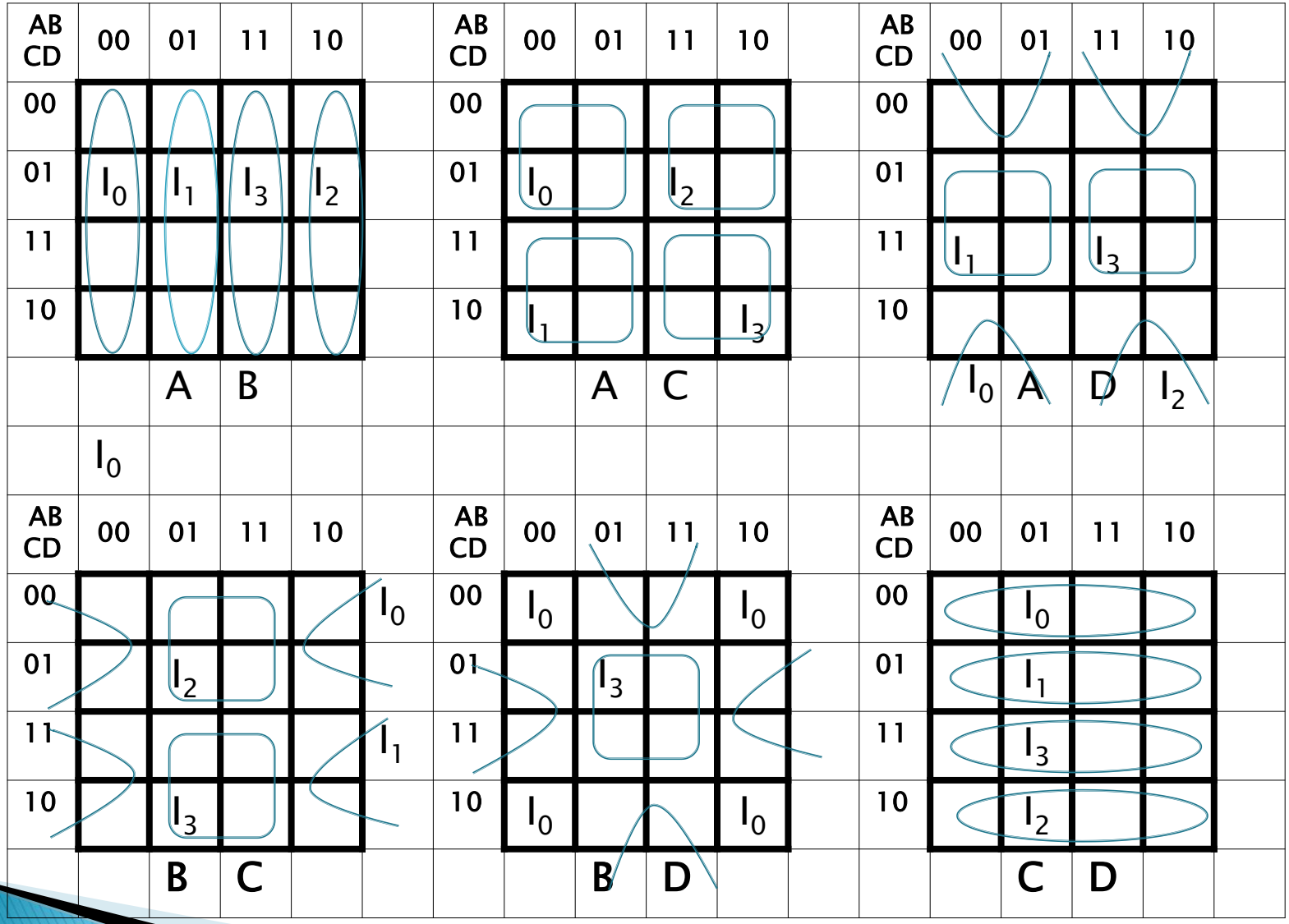
!x*t +
A*t + A*!x!z
+ B*z*!t



!x*t +
A*t + A*!x!z
+ B*z*!t

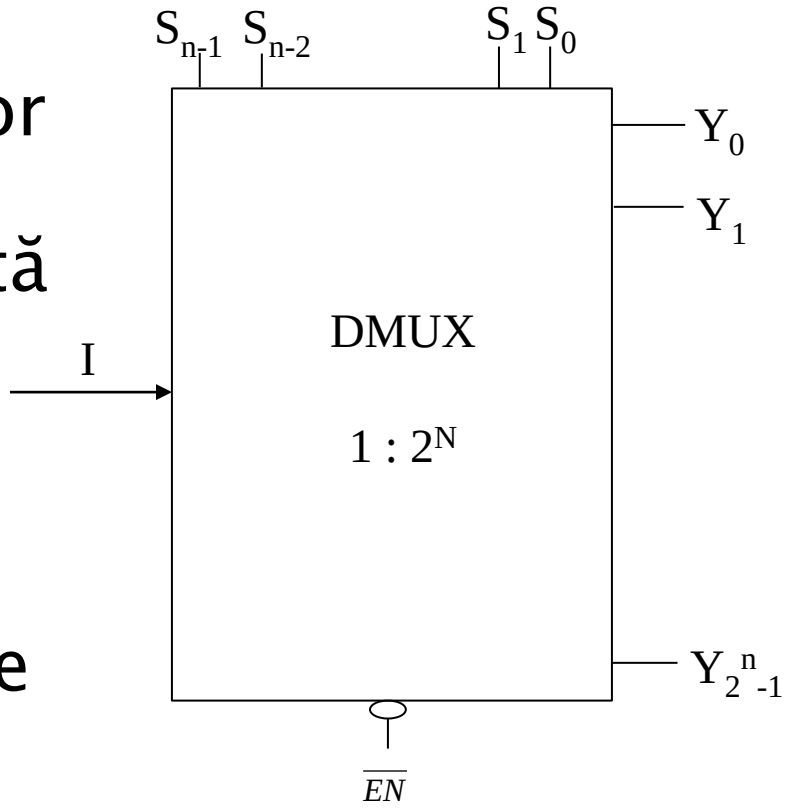


Spații de adresare și de selecție



Demultiplexorul

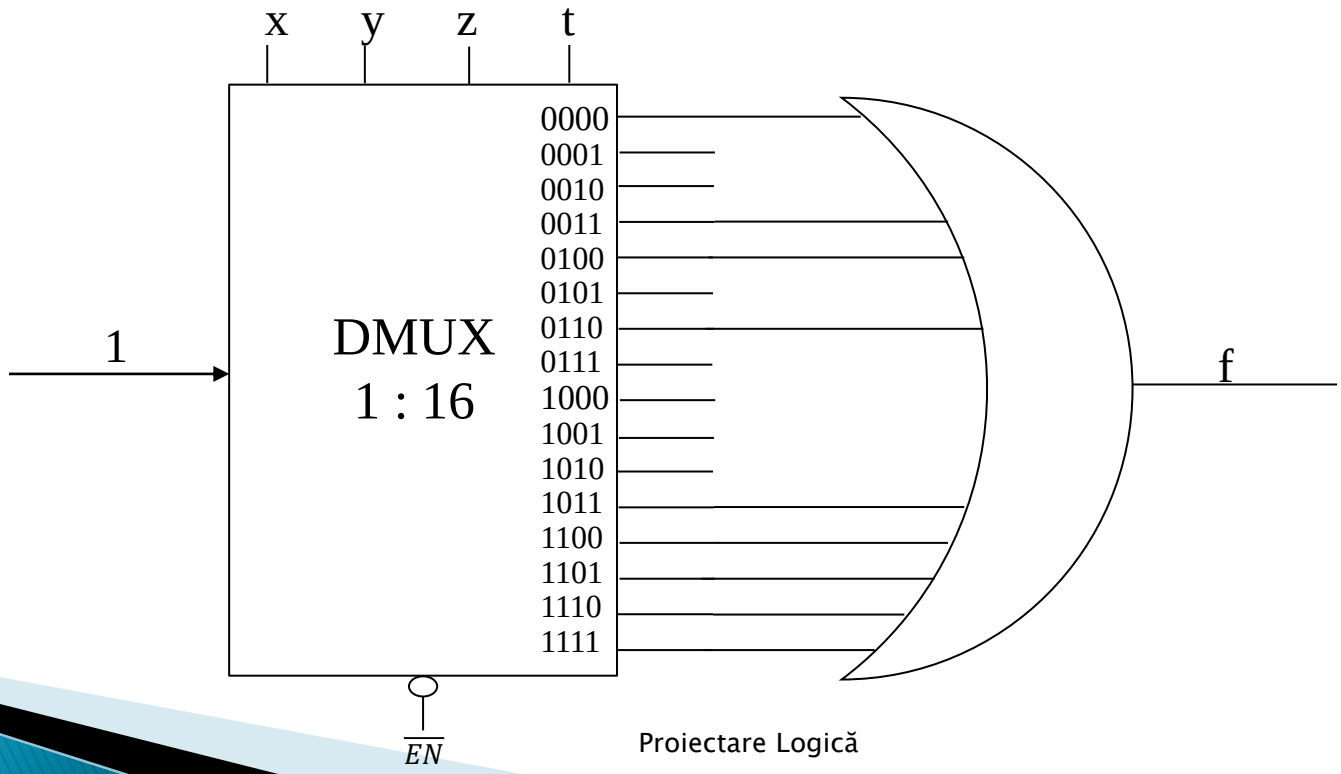
- Circuite complementare multiplexoarelor care permit transmiterea datelor de pe o intrare de date comună pe ieșirea selectată prin cod
- Un demultiplexor are o intrare de date (setată pe „1” logic), o intrare de enable, n intrări de selecție si 2^n ieșiri.
- $Y_k = I,\ k = (S_{n-1}\ldots S_0)_{10}$



x	y	z	t	f
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

Demultiplexoare – exemplu (1)

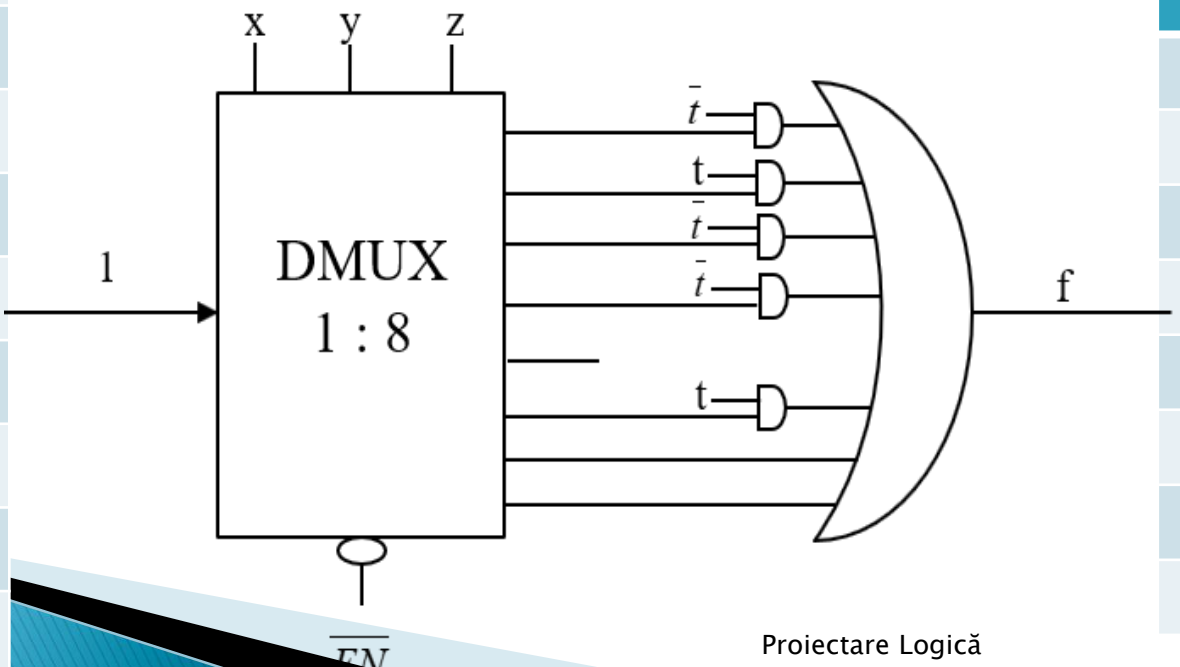
- Implementați funcția folosind DMUX 1:16
- Cum rezolvăm?
- Scriem funcția ca sumă de mintermi
- De ce să scriem funcția așa?
- Pentru sumă există element de circuit – poarta OR!
- Identificăm pentru ce combinații de valori funcția trebuie să ia valoarea „1” și punem ieșirile corespunzătoare într-o poartă de tip OR



Demultiplexoare – exemplu (2)

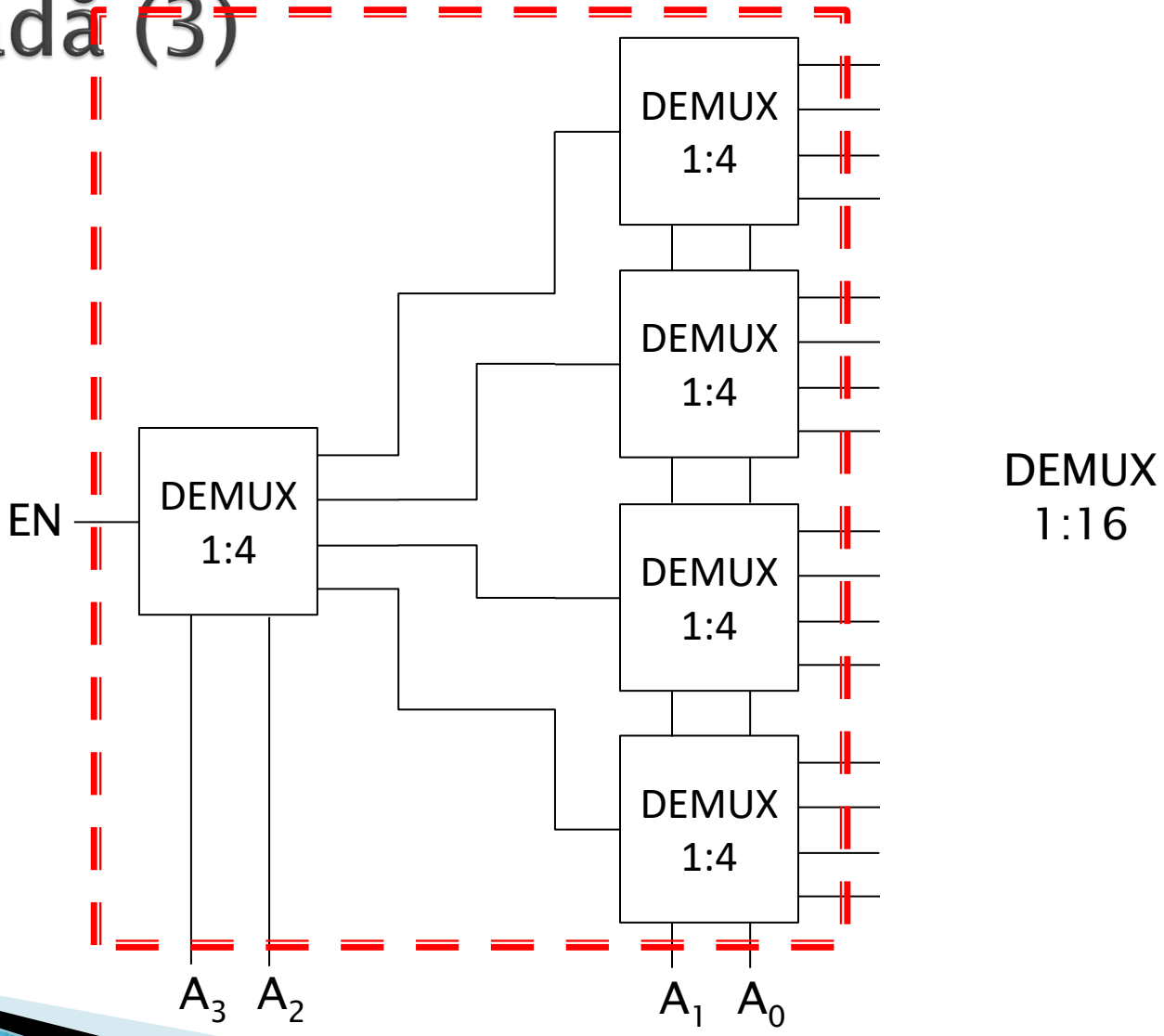
x	y	z	t	f
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

- Implementați funcția folosind DMUX 1:8
- Cum rezolvăm?
- Trecem o variabilă în coloana rezultat și apoi rezolvăm ca în cazul anterior
- Pentru realizarea valorilor ce depind de variabile, se intercalează între demultiplexor și poarta OR porți AND în care se pune variabila respectivă



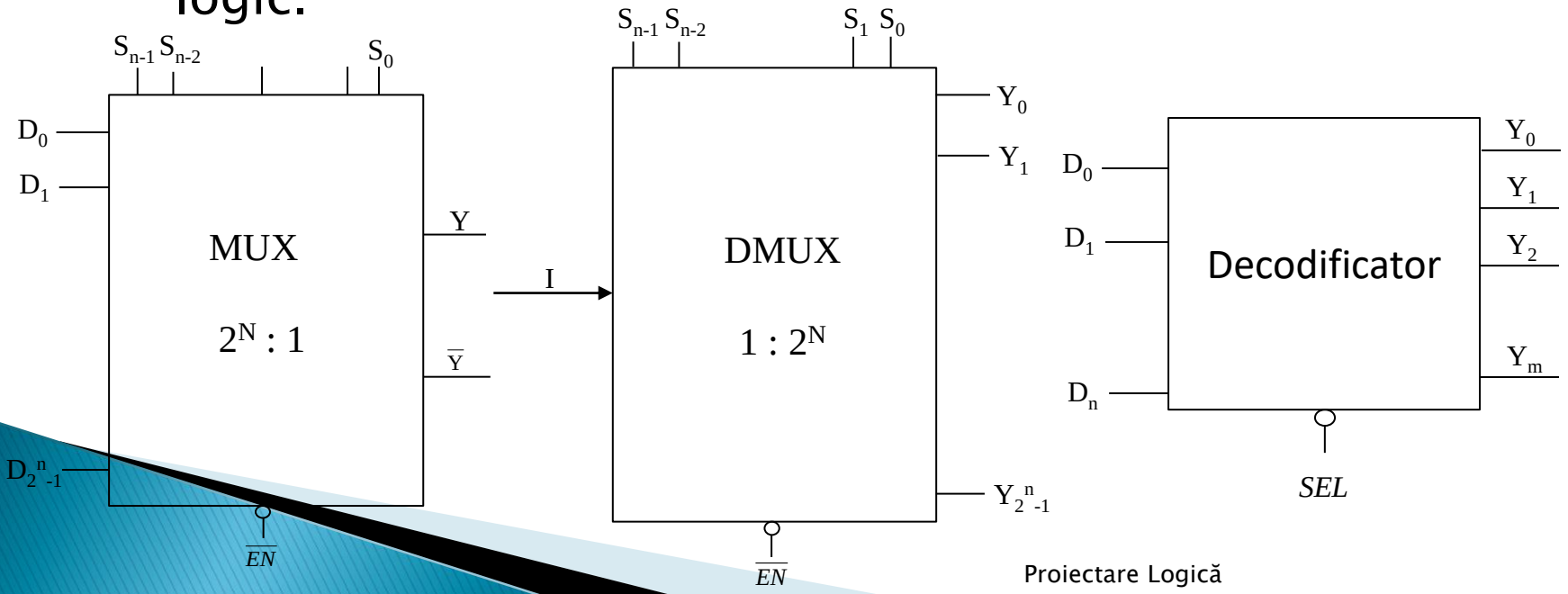
x	y	z	f
0	0	0	!t
0	0	1	t
0	1	0	!t
0	1	1	!t
1	0	0	0
1	0	1	t
1	1	0	1
1	1	1	1

Demultiplexoare – conectare în cascadă (3)



Codificatoare / Decodificatoare (1)

- Circuite combinaționale integrate pe scară medie
- Similare cu MUX/ DMUX
- Majoritatea lucrează în logică negativă (active pe 0)
  - În cazul unui decodificator semnalele de intrare aplicate căilor de adresare determină tranziţia uneia din ieşiri în 0 logic.



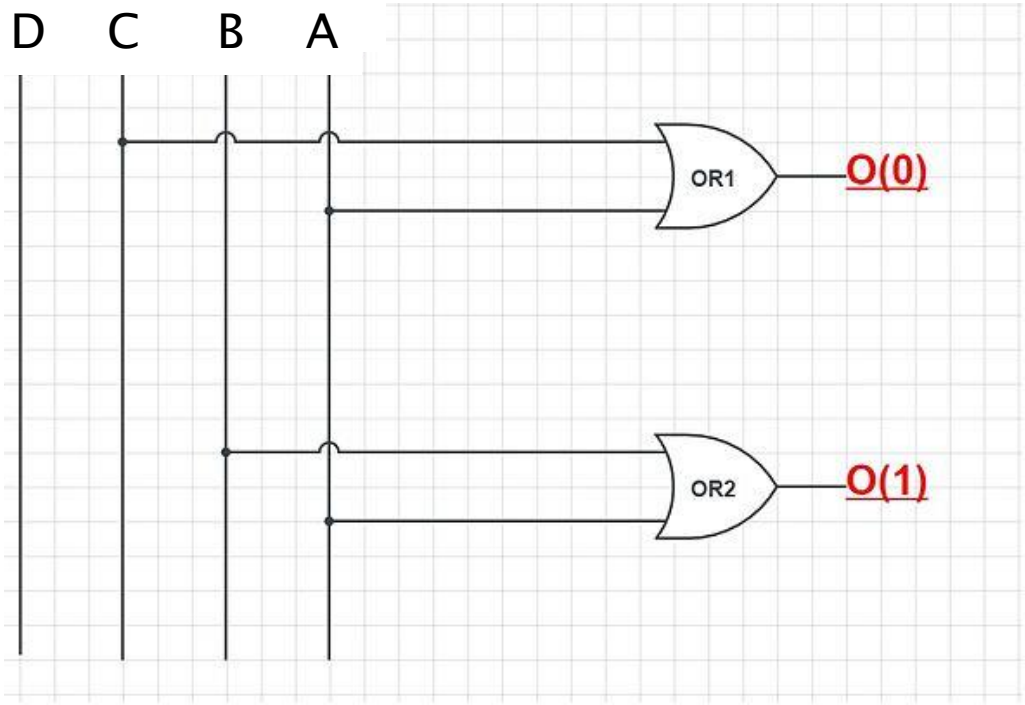
Codificatoare / Decodificatoare (2)

- *Codificatoare*: Vrem să obținem la ieșire codul binar al cifrei introduse la intrare
- Ex: la tastaturi (telefon)
- *Decodificatoare*: la intrare avem un anumit cod și la ieșire generează semnal pe o singură ieșire, cea corespunzătoare codului de la intrare (de la o reprezentare compactă a informației se obține una și doar una dintre porți activă)
- Ex: instalația de luminițe pentru brad când vrem să aprindem un anumit bec (serie de becuri) și nu alta / toate.

Codificatoare (1)

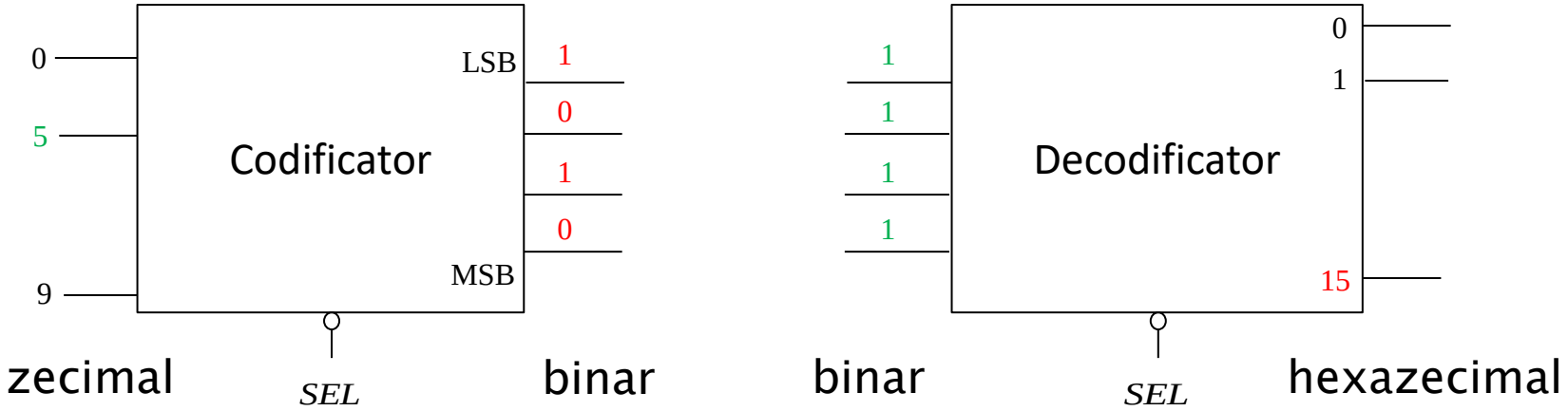
- Un **circuit codificator** atribuie o combinație de valori pentru ieșiri, unică, unui anumit semnal de intrare
- Doar o intrare are valoarea 1, la un moment dat
- Nu este obligatoriu o relație între numărul de intrări și numărul de ieșiri, deși în general avem relația $2^n - n$
- Exemplu codificator 4:2

A	B	C	D	O(1)	O(0)
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
Restul combinațiilor nu sunt permise				*	*



Codificatoare / Decodificatoare (3)

- E clar principiul de funcționare și diferența dintre ele (și MUX/DMUX)?



- Există o legătură între numărul de intrări și numărul de ieșiri:
  - Mux: 2^{sel} = intrări, Decodificatoare: $2^{intrări}$ = ieșiri
  - La codificatoare nu mai există intrări de selecție ci numai un semnal care îl lasă să funcționeze sau nu

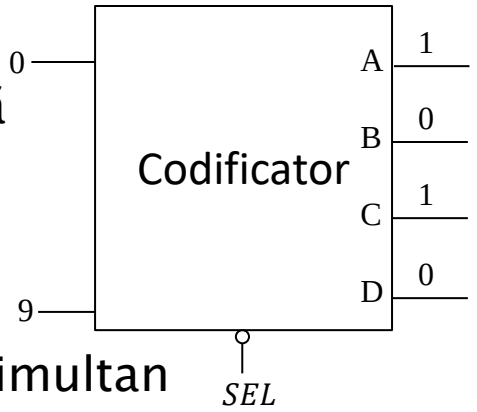
Codificatoare (2)

- Există codificatoare / decodificatoare binar – hexa, binar – zecimal (ultimele 6 ieșiri nu mai apar), binar –12
- În activitățile curente folosim baza 10 → avem nevoie de codificator zecimal-binar
  - Considerăm logica pozitivă (în cataloage sunt în logică negativă)

i_9	i_8	i_7	...	i_1	i_0	D	C	B	A
0	0	0	...	0	0	*	*	*	*
0	0	0	...	0	1	0	0	0	0
0	0	0	...	1	0	0	0	0	1
0	0	0	...	1	1	*	*	*	*

Câte linii în tabel?
Câte ne interesează
→invalidă

2^{10}
10



nu apăs 2 taste simultan (primesc semnal pe o singură intrare)

ÎNTREBĂRI?